

Java Backend Architecture

Interview Series

Chapter 13.4

Backpressure & Flow Control

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Introduction

Backpressure is the ability of a consumer to signal to a producer how much data it can handle, preventing the producer from overwhelming a slow consumer with more data than it can process. The Reactive Streams specification formalises this through the `request(n)` method on `Subscription`: a `Subscriber` explicitly pulls exactly `N` items at a time rather than being pushed an unlimited flood. Without backpressure, fast producers and slow consumers create unbounded queues that grow until `OutOfMemoryError` occurs. This is analogous to TCP's flow control window where the receiver advertises available buffer space to prevent the sender from transmitting more than the receiver can absorb.

Beyond reactive streams backpressure, flow control encompasses rate limiting (Bucket4j token bucket, Resilience4j `RateLimiter`), debounce and throttle patterns for event deduplication, bounded queues in thread pools, the bulkhead pattern for concurrency isolation, and Semaphore-based limiting. Understanding when to use BUFFER vs DROP vs LATEST vs ERROR overflow strategies is a critical design decision: BUFFER protects against transient spikes but risks OOM; DROP and LATEST discard data to stay within memory bounds; ERROR surfaces the problem immediately for explicit handling. Circuit breakers and backpressure interact: a tripped breaker should also propagate demand reduction upstream to prevent queue buildup behind the open circuit.

Key Concepts & Definitions

Backpressure	Flow-control mechanism where a slow consumer signals to a fast producer to reduce emission rate.
request(n)	Reactive Streams <code>Subscription</code> method; <code>Subscriber</code> requests exactly <code>N</code> items from the <code>Publisher</code> .
BUFFER strategy	Queue excess items in memory until the subscriber catches up; risks OOM on unbounded queues.
DROP strategy	Discard incoming items when the downstream cannot keep up; suitable when latest data matters more.
LATEST strategy	Keep only the most recently emitted item when the downstream is slow; good for real-time feeds.
ERROR strategy	Emit <code>OverflowException</code> immediately when demand is exceeded; makes overflow explicit and visible.
Token Bucket	Rate-limiting algorithm: bucket holds tokens (capacity), tokens refill at a fixed rate, each request consumes tokens.
Bucket4j	Java token-bucket rate-limiting library; supports in-memory, Redis, Hazelcast distributed buckets.
Resilience4j RateLimiter	Resilience4j module for fixed/sliding-window rate limiting with thread blocking or reactive support.
Debounce	Emits an item only after a quiet period with no new items; used to collapse bursts into single events.
Throttle / Sample	Emits at most one item per time window regardless of how many arrive; reduces emission rate.

Bulkhead	Resilience pattern isolating concurrency pools per feature to prevent one service from consuming all threads.
Semaphore	JDK concurrency primitive limiting concurrent access; used for bulkhead-style concurrency bounding.
Bounded Queue	Thread pool queue with a maximum size; rejects/throttles new work when full (ArrayBlockingQueue).
Circuit Breaker	Stops calls to a failing service; when open, fast-fails and reduces upstream producer demand.

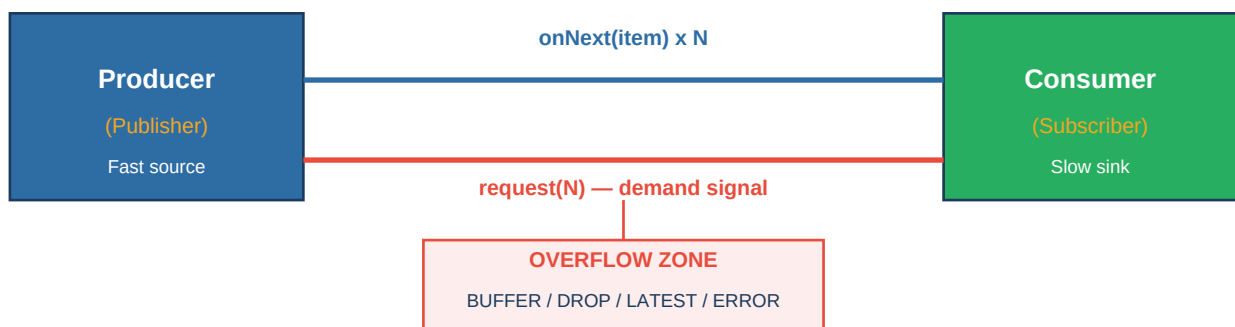
Overflow Strategy Comparison

Strategy	Operator	Behaviour	Data Loss	Memory Risk	Best For
BUFFER	onBackpressureBuffer()	Queue then drain	None	High	Transient spikes
DROP	onBackpressureDrop()	Discard new items	Yes (new)	Low	Latest-only feeds
LATEST	onBackpressureLatest()	Keep last item	Yes (old+new)	Very Low	Real-time prices
ERROR	onBackpressureError()	Throw exception	None until err	Low	Strict SLA systems
request(n)	BaseSubscriber	Pull-based demand	None	Controlled	Fine-grained control

Architecture Diagrams

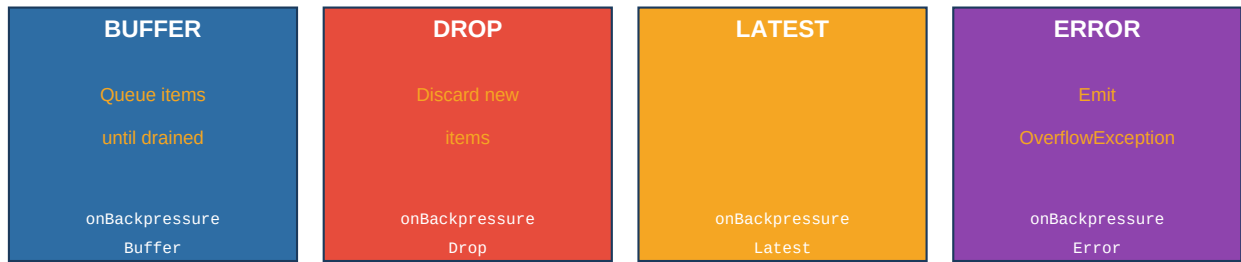
Reactive Streams Backpressure Protocol

Reactive Streams Backpressure Protocol



Flux Overflow Strategies

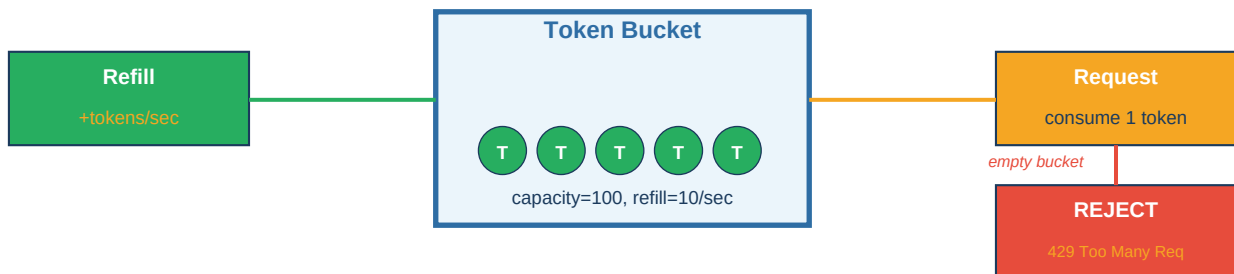
Flux Overflow / Backpressure Strategies



Default behavior when no strategy: **ERROR** (unbounded demand signals cause `IllegalStateException`)

Token Bucket Rate Limiting

Token Bucket Rate Limiting (Bucket4j)



Java Code Examples

1. Reactive Streams Manual Backpressure (BaseSubscriber)

```
Flux<Integer> source = Flux.range(1, 1000); source.subscribe(new BaseSubscriber<Integer>() {
    @Override protected void hookOnSubscribe(Subscription subscription) { log.info("Subscribed – requesting first 10"); request(10); // pull-based: request exactly 10 items }
    @Override protected void hookOnNext(Integer value) { process(value); if (value % 10 == 0) {
        log.info("Processed batch, requesting 10 more"); request(10); // request next batch after processing } } });
```

2. Flux Overflow Strategies

```
// Buffer with max size and DROP_OLDEST overflow
Flux<Event> safe = eventFlux
    .onBackpressureBuffer(500, dropped -> metrics.increment("events.dropped"),
        BufferOverflowStrategy.DROP_OLDEST); // Drop new items when slow subscriber can't keep up
Flux<Price> prices = tickStream .onBackpressureDrop(p -> log.warn("Price tick dropped: {}"),
    p.getSymbol()); // Keep only the latest – ideal for real-time dashboards
Flux<Metric> dashboard = metricsStream .onBackpressureLatest(); // Error immediately on overflow – strict systems
Flux<Transaction> transactions = txnStream .onBackpressureError();
```

3. Bucket4j Token Bucket Rate Limiter

```
// pom: com.github.vladimir-bukhtoyarov:bucket4j-core
Bandwidth limit = Bandwidth.classic(100,
    Refill.greedy(10, Duration.ofSeconds(1))); // 100 capacity, 10 tokens/sec
Bucket bucket = Bucket.builder().addLimit(limit).build();
@RestController public class ApiController {
    @GetMapping("/api/data") public ResponseEntity<Data> getData() { if (bucket.tryConsume(1)) {
        return ResponseEntity.ok(service.getData()); } return
        ResponseEntity.status(HttpStatus.TOO_MANY_REQUESTS); }
```

```
.header("X-Rate-Limit-Retry-After-Seconds", String.valueOf(bucket.getAvailableTokens()))
.build(); } }
```

4. Resilience4j RateLimiter (Reactive)

```
RateLimiterConfig config = RateLimiterConfig.custom() .limitForPeriod(50) // 50 calls per
refresh period .limitRefreshPeriod(Duration.ofSeconds(1))
.timeoutDuration(Duration.ofMillis(100)) // wait 100ms for a permission .build(); RateLimiter
rateLimiter = RateLimiter.of("myService", config); // Reactive integration Mono<Data> result =
Mono.fromCallable(() -> service.call())
.transformDeferred(RateLimiterOperator.of(rateLimiter))
.onErrorResume(RequestNotPermitted.class, ex -> Mono.error(new TooManyRequestsException()));
```

5. Semaphore Bulkhead (Concurrency Limiting)

```
// Limit concurrent DB calls to 20 regardless of thread pool size private final Semaphore
dbSemaphore = new Semaphore(20); public Data fetchFromDb(long id) { if
(!dbSemaphore.tryAcquire()) { throw new BulkheadFullException("DB bulkhead full"); } try {
return dbRepository.findById(id); } finally { dbSemaphore.release(); } } // Resilience4j
Bulkhead equivalent BulkheadConfig cfg = BulkheadConfig.custom() .maxConcurrentCalls(20)
.maxWaitDuration(Duration.ofMillis(50)) .build(); Bulkhead bulkhead = Bulkhead.of("db", cfg);
```

6. Debounce and Throttle in Reactor

```
// Debounce: emit only after 300ms of silence (e.g., search-as-you-type) Flux<String>
searchResults = keyPressFlux .debounce(Duration.ofMillis(300)) // waits 300ms quiet before
emitting .flatMap(query -> searchService.search(query)); // Throttle (sample): emit at most one
item per 1 second window Flux<Price> throttledPrices = priceStream
.sample(Duration.ofSeconds(1)); // emits latest item from each 1s window // throttleFirst: emit
FIRST item from each window, drop rest Flux<Click> uniqueClicks = clickStream
.throttleFirst(Duration.ofMillis(500), // ignore duplicate clicks within 500ms
Schedulers.parallel());
```

Interview Q&A;

Q1. What is backpressure and why is it important?

Backpressure is a flow-control mechanism allowing a slow consumer to signal a fast producer to reduce its emission rate. Without it, a fast producer overwhelms a slow consumer, building unbounded queues that cause `OutOfMemoryError`. The Reactive Streams spec implements backpressure via `Subscription.request(n)`: the subscriber explicitly pulls items rather than being passively pushed unlimited data.

Q2. How does Reactive Streams `request(n)` implement backpressure?

When a Subscriber subscribes to a Publisher, it receives a `Subscription`. It calls `subscription.request(n)` to signal demand for exactly `N` items. The Publisher may only emit up to `N` items (via `onNext`). When the Subscriber has processed those items and is ready for more, it calls `request(n)` again. This pull-based protocol prevents the Publisher from ever emitting more than the Subscriber has demanded.

Q3. What are the four Flux overflow strategies and when do you use each?

BUFFER queues excess items — use for transient spikes where data must not be lost, but watch for OOM. **DROP** discards new items when the subscriber is slow — use when latest data is irrelevant. **LATEST** keeps only the most recent item — best for real-time feeds (price ticks, sensor data). **ERROR** throws `OverflowException` immediately — use when any overflow indicates a design bug that must be surfaced.

Q4. What is the token bucket algorithm?

A token bucket maintains a bucket with a maximum capacity of tokens. Tokens are added at a fixed refill rate. Each incoming request consumes one (or more) tokens. If tokens are available, the request proceeds;

otherwise it is rejected (429 Too Many Requests) or queued. Unlike leaky bucket (which smooths output), token bucket allows bursting up to the bucket capacity while enforcing a long-term average rate.

Q5. How does Bucket4j implement rate limiting?

Bucket4j is a Java library implementing the token bucket algorithm. You define a Bandwidth (capacity + Refill — greedy or interval). `bucket.tryConsume(tokens)` returns true if tokens are available, false if rate-limited. It supports in-memory buckets and distributed buckets backed by Redis (Redisson), Hazelcast, or JDBC for multi-instance rate limiting. Also integrates with Spring via filter/interceptor.

Q6. What is the difference between debounce and throttle?

Debounce emits only after a specified quiet period with no new items — it delays emission until the stream is "stable." Use it for search-as-you-type to avoid a query per keystroke. Throttle (sample in Reactor) emits at most one item per time window, discarding the rest — it rate-reduces a high-frequency stream. `throttleFirst` emits the first item in each window; `sample` emits the latest.

Q7. What is the Bulkhead pattern and how does it relate to backpressure?

The Bulkhead pattern isolates concurrency pools by feature or service, preventing one slow service from consuming all threads. A Semaphore-based bulkhead caps concurrent calls at N; excess calls fail fast or queue briefly. This creates upstream backpressure: if the DB bulkhead is full, service-layer code receives rejection and can propagate that signal upstream (e.g., return 503 or reduce WebClient concurrency).

Q8. How does a circuit breaker interact with backpressure?

When a circuit breaker opens (service failing), it fast-fails all incoming calls. This should propagate as demand reduction upstream: rather than queuing calls behind an open circuit, the calling code should reduce its concurrency, increase delays, or shed load. In reactive pipelines, combine circuit breaker operators (Resilience4j CircuitBreakerOperator) with `onErrorResume` to shed load and prevent queue buildup behind the open circuit.

Q9. What is Resilience4j RateLimiter and how is it different from Bucket4j?

Resilience4j RateLimiter uses a fixed-window or sliding-window approach: it limits the number of calls permitted in a refresh period. It blocks or rejects calls that exceed the limit. Bucket4j uses token bucket which allows burst up to bucket capacity. Resilience4j integrates deeply with its other patterns (circuit breaker, bulkhead, retry) via a decorator chain; Bucket4j is simpler and offers distributed bucket support.

Q10. How do you implement backpressure in a Spring WebFlux controller?

Return `Flux<T>` from your controller. WebFlux propagates downstream demand from the HTTP client through the reactive chain. Configure `onBackpressureBuffer/Drop/Latest` as appropriate before expensive operators. For SSE streaming, Spring writes each item as it arrives; if the client reads slowly, TCP backpressure propagates through Netty's network buffers back up to the Flux. Never use `subscribeOn(boundedElastic)` to skip backpressure.

Q11. What is a bounded queue and why does it matter for flow control?

A bounded queue (e.g., `ArrayBlockingQueue`) has a maximum capacity. When full, producers block (for blocking queues) or are rejected. In thread pool executors, a bounded queue with `CallerRunsPolicy` causes the submitting thread to execute the task itself, naturally slowing the producer. Unbounded queues (`LinkedBlockingQueue`) allow unlimited backlog growth, risking OOM — the anti-pattern that backpressure is designed to prevent.

Q12. What happens in Reactor when you don't handle backpressure?

If you subscribe with simple lambda consumers (`subscribe(item -> ...)`) without requesting specific demand, Reactor implicitly requests `Long.MAX_VALUE` — unbounded demand. If the source is fast and the consumer

is slow, items build up in internal queues. For in-memory sources this is usually fine. For network or DB sources that respect demand, it means all data is requested immediately. Use `explicit request(n)` via `BaseSubscriber` for true flow control.

Q13. How does TCP flow control relate to application-level backpressure?

TCP uses a receive window (`rwnd`) — the receiver advertises how many bytes it can accept. The sender does not exceed the window. This is exactly the `request(n)` backpressure protocol at the TCP layer. In reactive systems, application-level backpressure (Reactive Streams) must work in concert with TCP flow control; if the application buffers too aggressively above TCP, it defeats TCP's own flow control and risks OOM.

Q14. How would you implement rate limiting in a reactive Spring Boot application?

Use a `WebFilter` that calls `bucket.tryConsumeAsync(1)` (`Bucket4j` async API) before routing. Return `Mono<Void>` from the filter. On token acquisition, call `chain.filter(exchange)`; on rate limit, write 429 response reactively. For distributed rate limiting (multiple instances), back the `Bucket` with Redis via `Bucket4j`'s Redisson integration, ensuring consistent rate enforcement across nodes.

Q15. What is the difference between `onBackpressureBuffer` with `DROP_OLDEST` vs `DROP_LATEST`?

When the buffer is full: `DROP_OLDEST` discards the oldest buffered item to make room for the new one — keeps the buffer current at the cost of losing old data. `DROP_LATEST` (or `DROP`) discards the incoming new item — keeps historical data in the buffer intact. `DROP_LATEST` is better when buffer contents represent an ordered sequence that must not be reordered; `DROP_OLDEST` is better when recency matters more.

Q16. How does Reactor handle backpressure for `Flux.interval()`?

`Flux.interval()` emits ticks on a `Scheduler` at a fixed rate regardless of downstream demand. If the downstream is slow, it cannot honor backpressure — it is a "hot" source with no demand awareness. Compose `onBackpressureLatest()` or `onBackpressureDrop()` downstream of `interval()` to explicitly choose an overflow strategy, otherwise a `MissingBackpressureException` is thrown at the subscriber.

Q17. What is reactive rate limiting with Spring Cloud Gateway?

Spring Cloud Gateway includes a Redis-backed rate limiter filter (`RequestRateLimiterGatewayFilterFactory`) using the token bucket algorithm. Configure via `application.yml` with `redis-rate-limiter.replenishRate` (tokens/sec) and `redis-rate-limiter.burstCapacity` (max bucket size). It uses Lua scripts on Redis for atomic token check-and-decrement, ensuring consistency across all Gateway instances.

Q18. How does the `Sample` operator differ from `Debounce`?

`sample(Duration)` emits the latest item from each fixed time window — it fires at regular intervals regardless of input rate, always taking the most recent available item. `debounce(Duration)` fires only after a quiet period with no new items — its timing depends on input gaps, not a fixed clock. `Sample` is for reducing a continuous stream to a manageable rate; `debounce` is for collapsing bursts after they stop.

Q19. What is the relationship between backpressure and memory?

Without backpressure, a fast publisher fills queues in memory indefinitely. Each unprocessed item occupies heap. Under sustained overload, the heap fills and GC pressure increases until OOM. With backpressure: `BUFFER` limits memory to `queue capacity * item size`; `DROP/LATEST` ensures only `O(1)` items are buffered; `ERROR` surfaces the problem before OOM. Proper backpressure strategy is the primary defense against memory-related outages under overload.

Q20. How do you test backpressure behavior with `StepVerifier`?

Use `StepVerifier.withVirtualTime()` for time-based operators. For demand testing, use `StepVerifier.create(flux, 0)` to start with zero demand, then call `thenRequest(n)` to simulate incremental demand and `expectNext(n)`

items). Use `expectError(MissingBackpressureException.class)` to assert overflow errors. This lets you verify that your flux correctly handles both slow and fast consumers in a deterministic, synchronous test.